Bài 1: Thiết lập định tuyến cho các thao tác với review trong ứng dụng minh hoạ.

Việc này cần thực hiện trong tệp tin movies.router.js

1.1 Định tuyến này sẽ có đường dẫn cuối cùng là '/review'

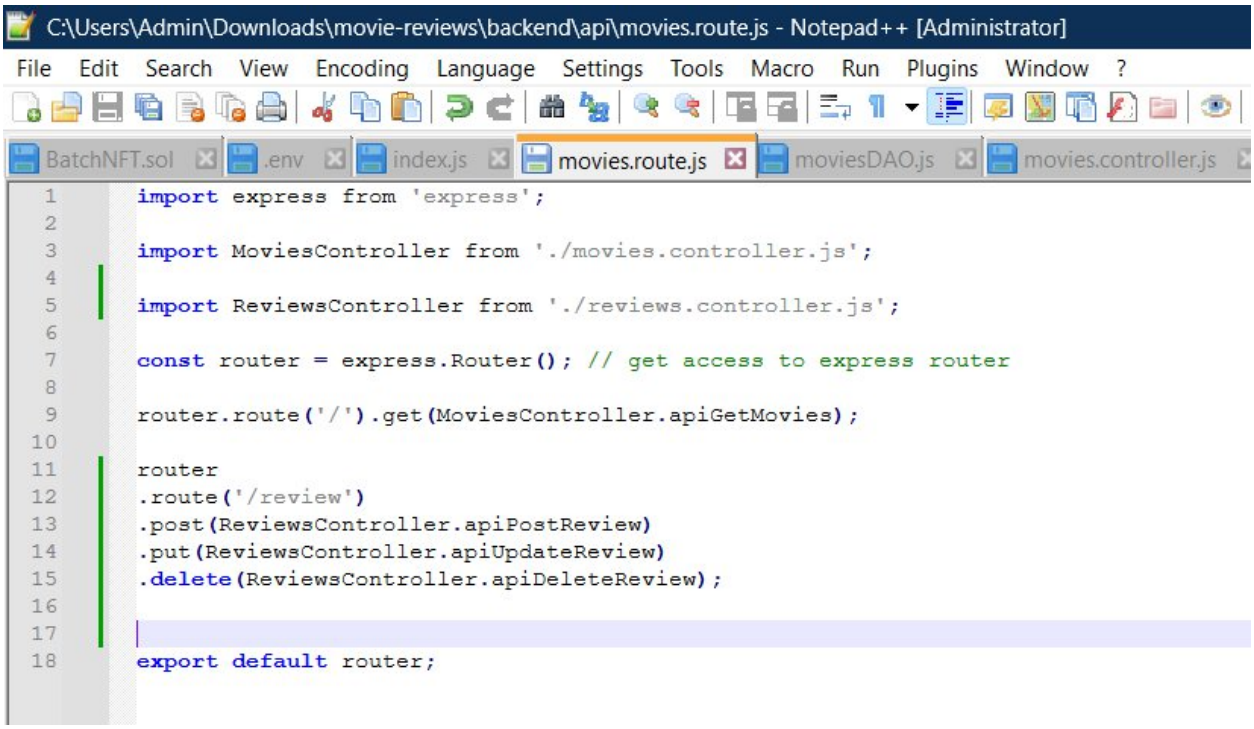
Ví dụ: localhost:3000/api/v1/movies/review

1.2 Thiết lập định tuyến thêm review vào db thông qua phương thức post, và gọi đến phương thức apiPostReview trong class ReviewsController trong tệp tin reviews.controller.js mà ta sẽ thiết lập ở bài 2.

1.3 Thiết lập định tuyến sửa review trên db thông qua phương thức put, và gọi đến phương

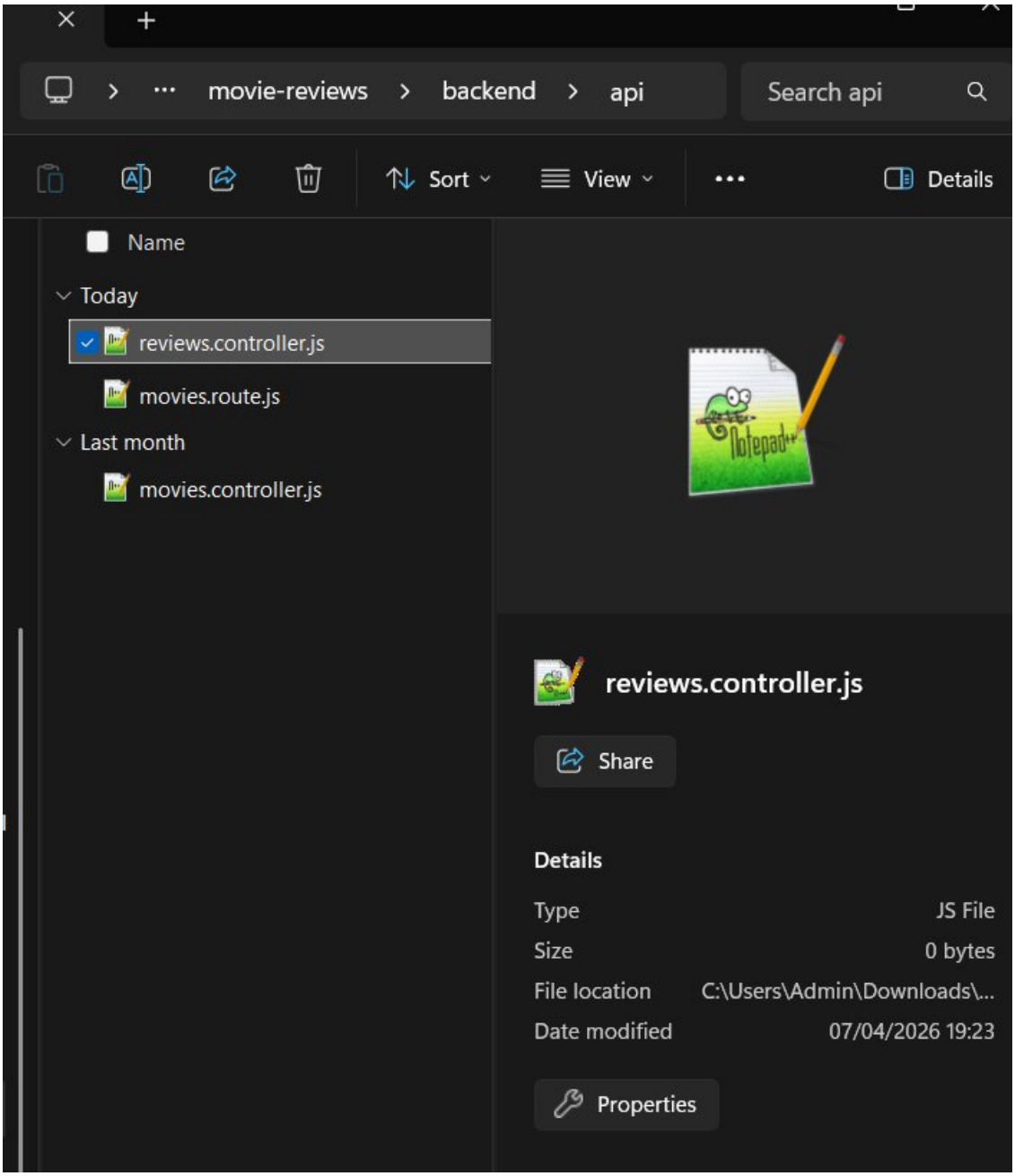
thức apiUpdateReview trong class ReviewsController trong tệp tin reviews.controller.js mà ta sẽ thiết lập ở bài 2.

1.4 Thiết lập định tuyến xoá review trên db thông qua phương thức delete, và gọi đến phương thức apiDeleteReview trong class ReviewsController trong tệp tin reviews.controller.js mà ta sẽ thiết lập ở bài 2.



Bài 2: Thiết lập Controller cho review.

2.1 Tạo tệp tin reviews.controller.js trong thư mục api chứa một class có tên ReviewsController để quản lý các yêu cầu có liên quan đến review từ người dùng gửi lên từ máy khách.



Ta tạo tệp tin reviews.controller.js trong thư mục api

2.2 Trong tệp tin vừa tạo ở bài 2.1 sẽ chứa dòng lệnh import nội dung từ tệp tin reviewsDAO.js

(sẽ tạo ở bài 3) để gọi tới các hàm tương tác dữ liệu.

2.3 Tạo phương thức có tên apiPostReview() để quản lý các yêu cầu được gửi từ máy khách

(thiết lập ở bài 1.2).

- Phương thức này sẽ lấy dữ liệu gửi lên từ người dùng thông qua tham số req gồm có

movie_id, review, userinfo gồm name và id (gửi dưới dạng JSON trong body của req), và tạo

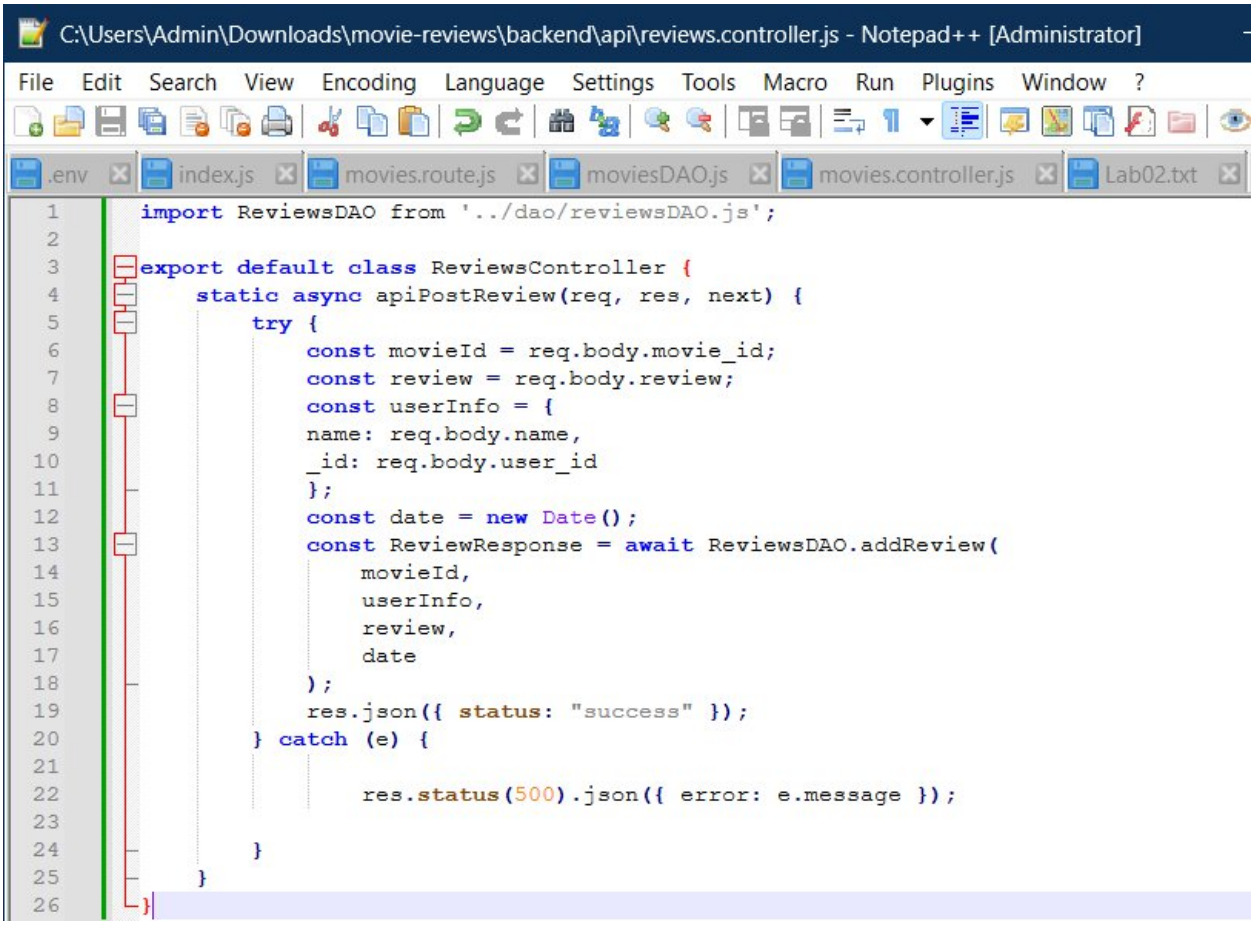
ra một biến date để lưu trữ ngày tháng năm hiện tại của review.

- Sau đó, gọi đến hàm addReview() được định nghĩa trong ReviewsDAO (sẽ tạo ở bài 3)

để thêm được review vào db.

- Nếu thêm dữ liệu thành công sẽ trả về cho máy khách 1 thông báo dưới dạng JSON

báo 'success', nếu không thành công sẽ log lỗi ra trên màn hình console của terminal.




```
import ReviewsDAO from '../dao/reviewsDAO.js';

export default class ReviewsController {
    static async apiPostReview(req, res, next) {
        try {
            const movieId = req.body.movie_id;
            const review = req.body.review;
            const userInfo = {
            name: req.body.name,
            _id: req.body.user_id
            };
            const date = new Date();
            const ReviewResponse = await ReviewsDAO.addReview(
                movieId,
                userInfo,
                review,
                date
            );
            res.json({ status: "success" });
        } catch (e) {

                res.status(500).json({ error: e.message });

        }
    }
}
```

2.4 Tạo phương thức có tên apiUpdateReview() để quản lý các yêu cầu được gửi từ máy khách

(thiết lập ở bài 1.3).

- Phương thức này sẽ lấy dữ liệu gửi lên từ người dùng thông qua tham số req gồm có

review_id, user_id (gửi dưới dạng JSON trong body của req), và tạo ra một biến date để lưu

trữ ngày tháng năm hiện tại của review mới, lưu ý, muốn update thành công review thì id user phải là user đã tạo ra review.

- Sau đó, gọi đến hàm updateReview() được định nghĩa trong ReviewsDAO (sẽ tạo ở bài

3) để sửa dữ liệu review trên db.

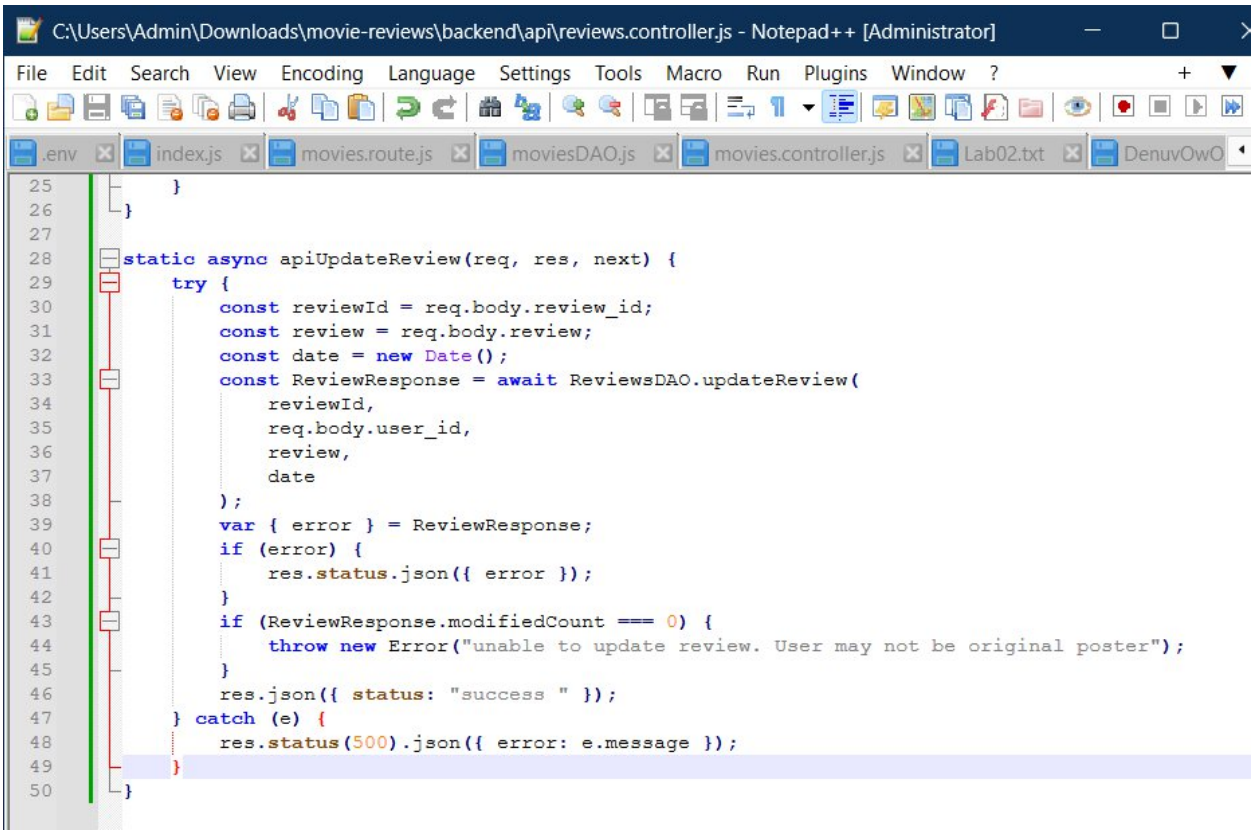
- Nếu thêm dữ liệu thành công sẽ trả về cho máy khách 1 thông báo dưới dạng JSON

báo ‘success’, nếu không thành công sẽ log lỗi ra trên màn hình console của terminal.

- Lưu ý: Cần tạo ra biến ReviewResponse để lấy kết quả trả về khi gọi hàm

updateReview(), vì hàm updateReview() trong bài 3 sẽ có trả về một biến tên là modifiedCount

để xác định xem là có review nào thực sự đã được chỉnh sửa hay chưa.



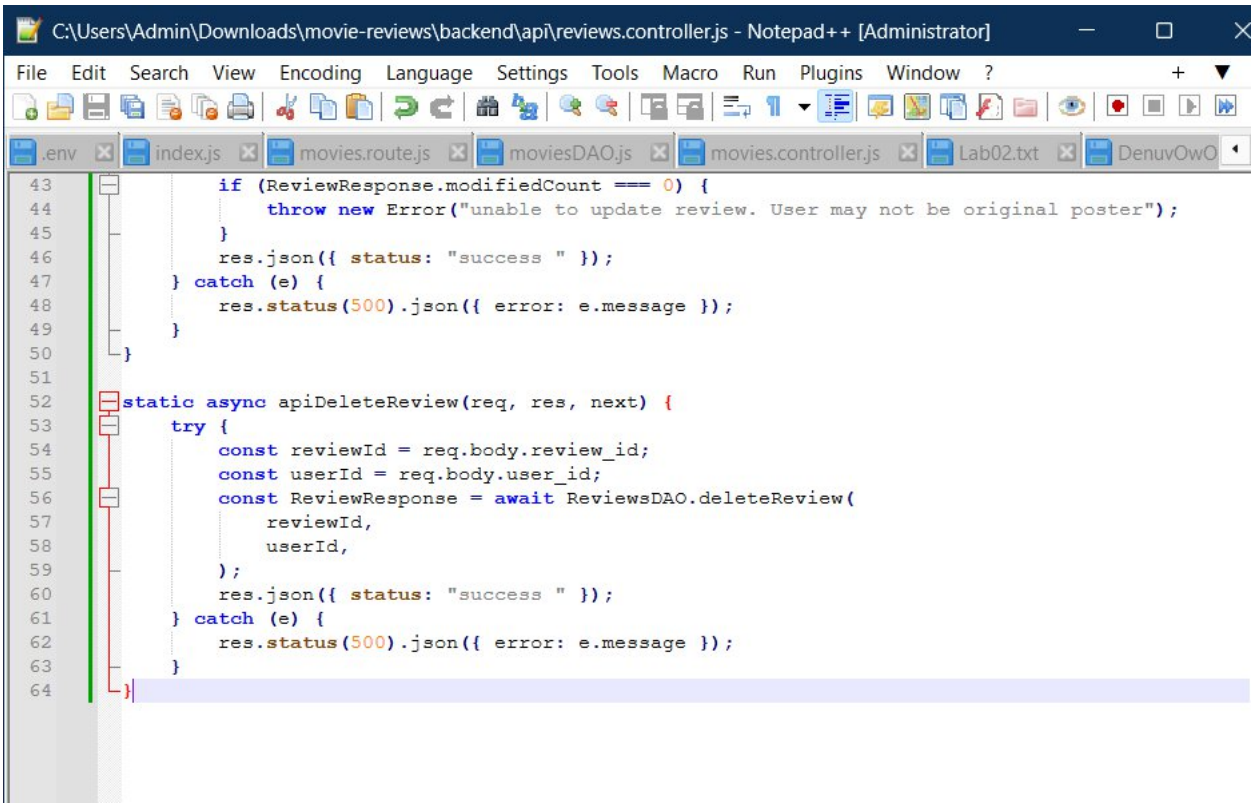

```
25      }
26  }
27
28  static async apiUpdateReview(req, res, next) {
29      try {
30          const reviewId = req.body.review_id;
31          const review = req.body.review;
32          const date = new Date();
33          const ReviewResponse = await ReviewsDAO.updateReview(
34              reviewId,
35              req.body.user_id,
36              review,
37              date
38          );
39          var { error } = ReviewResponse;
40          if (error) {
41              res.status.json({ error });
42          }
43          if (ReviewResponse.modifiedCount === 0) {
44              throw new Error("unable to update review. User may not be original poster");
45          }
46          res.json({ status: "success " });
47      } catch (e) {
48          res.status(500).json({ error: e.message });
49      }
50  }
```

2.5 Tạo phương thức có tên apiDeleteReview() để quản lý các yêu cầu được gửi từ máy khách

(thiết lập ở bài 1.3).

- Phương thức này sẽ lấy dữ liệu gửi lên từ người dùng thông qua tham số req gồm có review_id, user_ id (gửi dưới dạng JSON trong body của req), và thực hiện việc xoá review

thông qua hàm deleteReview trong lớp ReviewsDAO sẽ tạo ở bài 3.



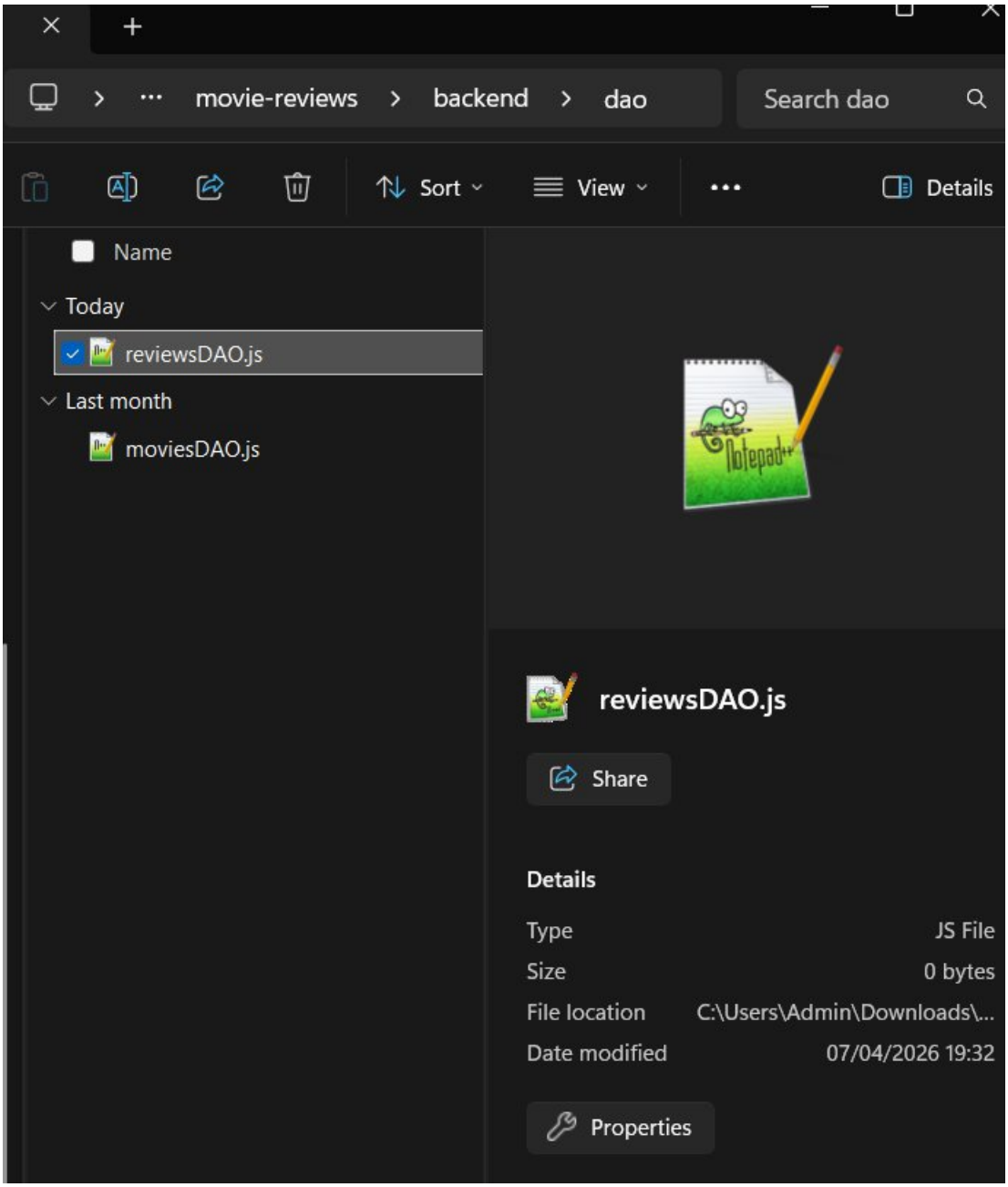
Bài 3: Thiết lập DAO cho reviews.

3.1 Trong thư mục DAO tạo tệp tin reviewsDAO.js.

- Tệp tin này cần import pakage mongodb đã cài đặt ở lab 2 để sử dụng một số phương thức cần thiết.

- Tạo một hằng số tên ObjectId = mongodb.ObjectId để sau này xử lý một số tác vụ liên quan đến trường dữ liệu _id trong mongodb.

- Tạo ra một biến reviews để tham chiếu tới collection reviews sẽ tạo sau trên db.



Ta tạo tệp tin reviewsDAO.js

3.2 Tạo phương thức có tên injectDB() giúp kết nối tới collection tương ứng trên db.

- Cần tạo đối tượng và gọi phương thức injectDB() này trong tệp tin index.js để đảm bảo kết nối tới collection thành công, nhằm mục đích tương tác với dữ liệu.

- Lưu ý: việc gọi injectDB() này trong tệp tin index phải sau dòng lệnh kết nối tới dữ liệu và trước khi khởi tạo máy chủ web.

C:\Users\Admin\Downloads\movie-reviews\backend\dao\reviewsDAO.js - Notepad++ [Administrator]

```js
import mongodb from "mongodb";
const ObjectId = mongodb.ObjectId;
let reviews;
export default class ReviewsDAO {
    static async injectDB(conn) {
        if (reviews) {
                return;
        }
        try {
                reviews = await conn.db(process.env.MOVIEREVIEWS_NS).collection('reviews');
        }
        catch (e) {
                console.error(`unable to establish connection handle in reviewDAO: ${e}`);
        }
    }
}
```

*C:\Users\Admin\Downloads\movie-reviews\backend\index.js - Notepad++ [Administrator]

```js

import MoviesDAO from "./dao/moviesDAO.js";

import ReviewsDAO from "./dao/reviewsDAO.js"

async function main() {
dotenv.config();
const client = new
mongodb.MongoClient(process.env.MOVIEREVIEWS_DB_URI);
const port = process.env.PORT || 8000;
try {
// Connect to the MongoDB cluster
    await client.connect();

    await MoviesDAO.injectDB(client);

    await ReviewsDAO.injectDB(client);

    app.listen(port, () => {
        console.log('Server is running on port: ' + port);
    });
    } catch (e) {
console.error(e);
process.exit(1);
};
}
main().catch(console.error);
```

3.3 Tạo phương thức addReview() để thêm review vào db, trong hàm này sẽ có gọi một hàm

insertOne(), lưu ý phải biến chuỗi movieId trong tham số truyền vào ở bài 2 thành dạng ObjectId.

C:\Users\Admin\Downloads\movie-reviews\backend\dao\reviewsDAO.js - Notepad++ [Administrator]

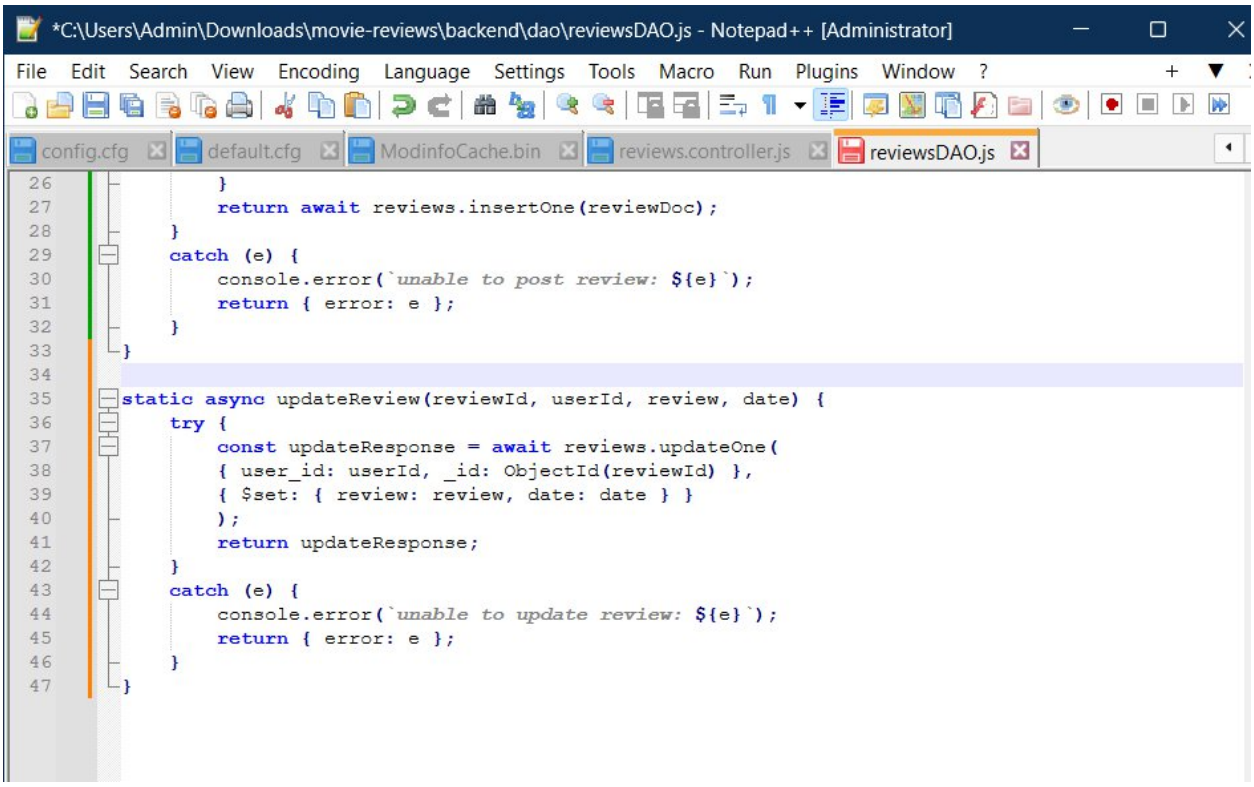
```
        }
        try {
                reviews = await conn.db(process.env.MOVIEREVIEWS_NS).collection('reviews');
        }
        catch (e) {
                console.error(`unable to establish connection handle in reviewDAO: ${e}`);
        }
    }
}

static async addReview(movieId, user, review, date) {
    try {
        const reviewDoc = {
            name: user.name,
            user_id: user._id,
            date: date,
            review: review,
            movie_id: ObjectId(movieId)
        }
        return await reviews.insertOne(reviewDoc);
    }
    catch (e) {
        console.error(`unable to post review: ${e}`);
        return { error: e };
    }
}
```

Thêm phương thức addreview() vào reviewsDAO.js

3.4 Tạo phương thức updateReview() để sửa review trên db, trong hàm này sẽ có gọi một hàm

updateOne(), lưu ý phải biến chuỗi reviewId trong tham số truyền vào ở bài 2 thành dạng

ObjectId, phải cùng userId mới cho phép sửa review.



Thêm phương thức updatereview() vào reviewsDAO.js

3.5 Tạo phương thức deleteReview() để thêm review vào db, trong hàm này sẽ có gọi một hàm

deleteOne(), lưu ý phải biến chuỗi reviewId trong tham số truyền vào ở bài 2 thành dạng

ObjectId, phải cùng userId mới cho phép xoá review.

C:\Users\Admin\Downloads\movie-reviews\backend\dao\reviewsDAO.js - Notepad++ [Administrator]

```
39            { $set: { review: review, date: date } }
40            );
41            return updateResponse;
42        }
43        catch (e) {
44            console.error(`unable to update review: ${e}`);
45            return { error: e };
46        }
47    }
48
49    static async deleteReview(reviewId, userId) {
50        try {
51            const deleteResponse = await reviews.deleteOne({
52                _id: ObjectId(reviewId),
53                user_id: userId,
54            });
55            return deleteResponse;
56        }
57        catch (e) {
58            console.error(`unable to delete review: ${e}`);
59            return { error: e };
60        }
61    }
```

3.6 Thử nghiệm các API xem đã thành công hay chưa bằng phần mềm hỗ trợ như Insomnia:

Yêu cầu đổi user_id thành MSSV

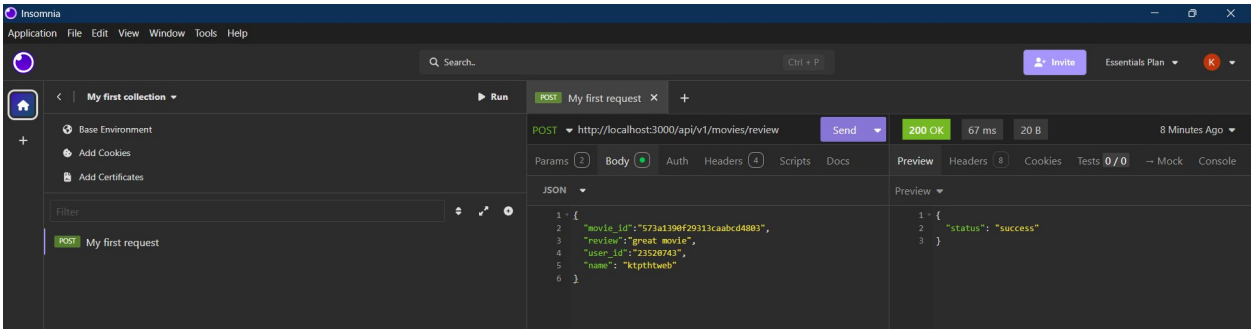
Ví dụ:

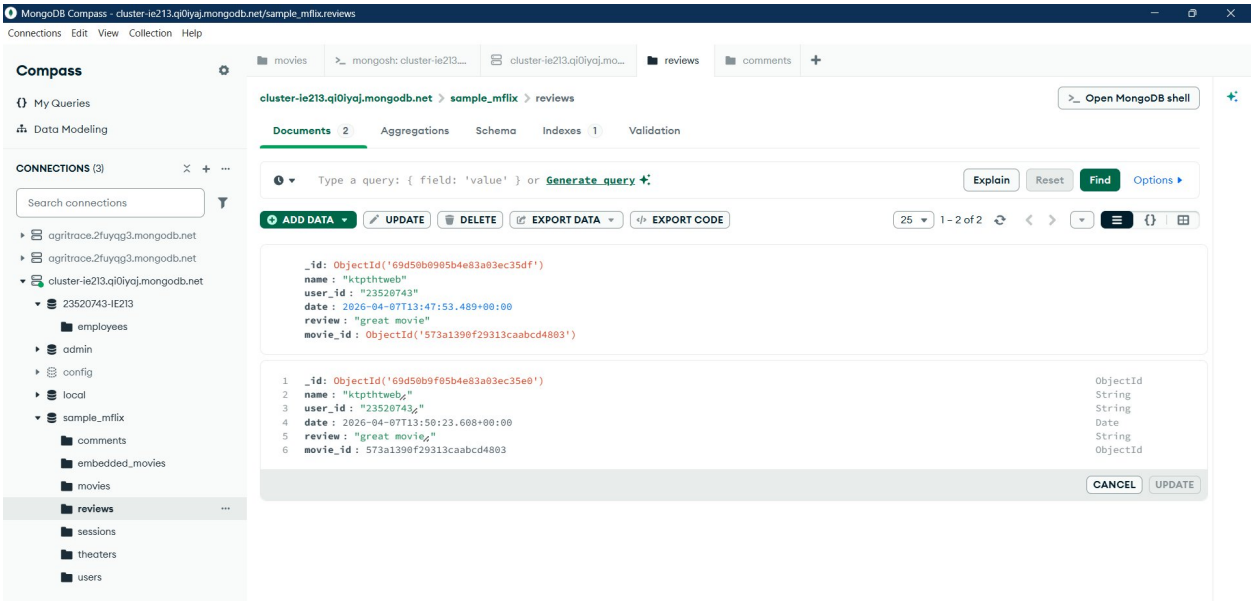
Thêm/Xoá/Sửa dữ liệu (tham khảo slide 60-63 trong bài học)

Administrator: Windows Powe

```
    at afterResolve (node:internal/modules/esm/loader:625:52)
    at ModuleLoader.getOrCreateModuleJob (node:internal/modules/esm/loader:631:12)
    at ModuleJob.syncLink (node:internal/modules/esm/module_job:160:33)
    at ModuleJob.link (node:internal/modules/esm/module_job:245:17)

Node.js v24.14.0
PS C:\Users\Admin\Downloads\movie-reviews\backend> node server.js
file:///C:/Users/Admin/Downloads/movie-reviews/backend/dao/reviewsDAO.js:18
static async addReview(movieId, user, review, date) {
^^^^^^

SyntaxError: Unexpected strict mode reserved word
    at compileSourceTextModule (node:internal/modules/esm/utils:318:16)
    at ModuleLoader.moduleStrategy (node:internal/modules/esm/translators:99:18)
    at #translate (node:internal/modules/esm/loader:473:20)
    at afterLoad (node:internal/modules/esm/loader:529:29)
    at ModuleLoader.loadAndTranslate (node:internal/modules/esm/loader:534:12)
    at #getOrCreateModuleJobAfterResolve (node:internal/modules/esm/loader:577:36)
    at afterResolve (node:internal/modules/esm/loader:625:52)
    at ModuleLoader.getOrCreateModuleJob (node:internal/modules/esm/loader:631:12)
    at ModuleJob.syncLink (node:internal/modules/esm/module_job:160:33)
    at ModuleJob.link (node:internal/modules/esm/module_job:245:17)

Node.js v24.14.0
PS C:\Users\Admin\Downloads\movie-reviews\backend> node server.js
PS C:\Users\Admin\Downloads\movie-reviews\backend> node server.js
PS C:\Users\Admin\Downloads\movie-reviews\backend> node server.js
 Server running on port 3000
```

Ta chạy và đảm bảo server chạy





Test POST review bằng insomnia, ta thấy server báo thành công và sau đó đối chiếu với csdl trên mongo

Bài 4: Hoàn thành back-end cho ứng dụng minh họa.

4.1 Thêm 2 định tuyến cho người dùng sử dụng các chức năng sau:

1. Lấy tất cả thông tin của phim và các review có liên quan dựa trên Id của phim.

2. Lấy tất cả các loại rating của phim trên dữ liệu.

C:\Users\Admin\Downloads\movie-reviews\backend\api\movies.route.js - Notepad++ [Administrator]

```js
import express from 'express';

import MoviesController from './movies.controller.js';

import ReviewsController from './reviews.controller.js';

const router = express.Router(); // get access to express router

router.route('/').get(MoviesController.apiGetMovies);

router
.route('/review')
.post(ReviewsController.apiPostReview)
.put(ReviewsController.apiUpdateReview)
.delete(ReviewsController.apiDeleteReview);

router.route("/id/:id").get(MoviesController.apiGetMovieById);
router.route("/ratings").get(MoviesController.apiGetRatings);


export default router;
```

Thêm 2 định tuyến trong tệp tin movies.route.js

4.2 Thêm 2 phương thức controller tương ứng cho phần 4.1 là apiGetMovieById() và apiGetRatings() trong movie controller.

*C:\Users\Admin\Downloads\movie-reviews\backend\api\movies.controller.js - Notepad++ [Administrator]

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

.env | index.js | movies.route.js | moviesDAO.js | movies.controller.js | default.cfg | ModinfoC

```
28      res.json(response);
29  }
30
31  static async apiGetMovieById(req, res, next) {
32      try {
33          let id = req.params.id || {};
34          let movie = await MoviesDAO.getMovieById(id);
35          if (!movie) {
36              res.status(404).json({ error: "not found" })
37              return;
38          }
39          res.json(movie);
40      }
41      catch (e) {
42          console.log(`api, ${e}`);
43          res.status(500).json({ error: e });
44      }
45  }
46
47  static async apiGetRatings(req, res, next) {
48      try {
49          let propertyTypes = await MoviesDAO.getRatings();
50          res.json(propertyTypes);
51      }
52      catch (e) {
53          console.log(`api,${e}`);
54          res.status(500).json({ error: e });
55      }
56  }
57
58  }
```

4.3 Thêm 2 phương thức DAO tương ứng cho phần 4.2 là getRatings() và getMovieById() trong

dao movie.

Lưu ý: phần getMovieById() sẽ sử dụng một số toán tử và phương thức đã học ở bài thực hành

1 MongoDB như $match, $lookup (giống khóa ngoại trong SQL) và aggregate() để tổng hợp

dữ liệu từ nhiều collection.

*C:\Users\Admin\Downloads\movie-reviews\backend\dao\moviesDAO.js - Notepad++ [Administrator]

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

.env | index.js | movies.route.js | moviesDAO.js | movies.controller.js | default.cfg | ModinfoC

```
52
53    static async getMovieById(id) {
54      try {
55          return await movies.aggregate([
56          {
57              $match: { _id: new ObjectId(id),}
58          },
59          {
60              $lookup:{from: 'reviews',localField: '_id',
61                  foreignField: 'movie_id',as: 'reviews',
62
63              }
64          }
65          ]).next();
66      }
67      catch (e) {
68              console.error(`something went wrong in getMovieById: ${e}`);
69              throw e;
70      }
71  }
72
73    static async getRatings() {
74      let ratings = []
75      try {
76          ratings = await movies.distinct("rated");
77          return ratings;
78      }
79      catch (e) {
80          console.error(`unable to get ratings,
81          $(e)`);
82          return ratings;
83      }
84  }
85
86  }
```

4.4 Thử nghiệm các API vừa tạo ở trên.

Insomnia

Application File Edit View Window Tools Help

My first collection | Run | GET My first request

Base Environment
Add Cookies
Add Certificates

GET My first request

GET http://localhost:3000/api/v1/movies/ratings | Send | 200 OK | 78 ms | 125 B | Just Now

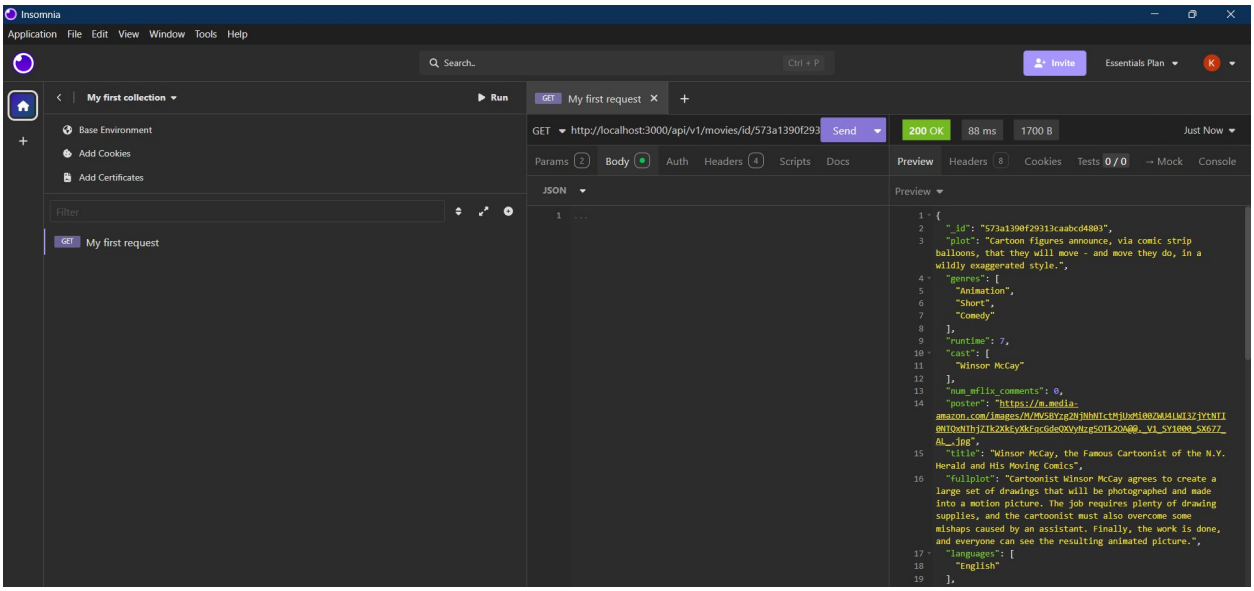
Params 2 | Body | Auth | Headers 4 | Scripts | Docs | Preview | Headers 8 | Cookies | Tests 0 / 0 | → Mock | Console

JSON

```
1 {
2   "review_id":"60d50b0905b4e83a03ec35df",
3   "user_id":"23520743"
4 }
```

Preview

```
1 [
2   "AO",
3   "APPROVED",
4   "Approved",
5   "G",
6   "GP",
7   "M",
8   "Not Rated",
9   "OPEN",
10  "PASSED",
11  "PG",
12  "PG-13",
13  "R",
14  "TV-14",
15  "TV-G",
16  "TV-MA",
17  "TV-PG",
18  "TV-Y7"
19 ]
```



Ta thử lệnh như get movie bằng id và thấy insomnia trả về kết quả là movie cần tìm